TIVIDY: Workflow Management System

Some Workflow

Group Leader:

Nasiha Osman

Team members:

Yasar Rahman - u24596397

Shanya Nair - u25061845

Nasiha Osman - u25110188

Tamima Alam Touhid - u24691284

Mohammed Lutchka - u25588304

Mueez Gani - u24983439

Tshegofatso Mahlangu - u23605032

Github Repository
https://github.com/shanya-06/TIVIDY-214_Project

Task 1

What a workflow system is. IBM describes a workflow as a way of managing repetitive tasks that happen in a set order. Workflow management systems route tasks to the right employee at the right time and pass along the information they need. [1] The Workflow Management Coalition (WfMC) puts it more formally. A workflow management system defines, manages and executes workflows, with the order of work driven by a computer model of the workflow logic. [2]

What the engine does. The core of such a system is the workflow engine. It tracks the state of each activity in a workflow, such as a loan application being processed and approved, and decides which activity comes next based on the defined process. [3] It also hands tasks to different people and passes data between them. [3] This is the job TIVIDY's main classes will do.

Definition versus instance. The workflow model is the template, and each run of it is a separate instance. A workflow instance is one execution of a workflow model. It holds the data for that run and its current state, and it is made of node instances in the same way the model is made of nodes. [4] So one definition can create many instances, and changing one instance does not change the others.

Common building blocks. The Workflow Patterns Initiative collected the requirements that keep coming up when processes are modelled. Practitioners, vendors and academics have used these patterns to select, design and build workflow systems. [5] The patterns cover control flow, data and resources (who does the work). This tells us that things like sequences, parallel work, choices, loops and task assignment belong in a reusable engine and should not be hard-coded for one process.

What this means for TIVIDY. Across these sources, a few responsibilities show up every time and can change independently: the lifecycle of work items, the rules for routing and assigning work, the connections to outside systems, and the history and reporting. We will keep these separate in our design. The engine stays generic, and the insurance claim is just one workflow that runs on it.

References

1. IBM (n.d.) *What is a workflow?* Available at: https://www.ibm.com/topics/workflow

2. Nuxeo (2024) *Glossary: Workflow Instance*. Nuxeo Documentation. Available at: https://doc.nuxeo.com/n/WPY

3. Russell, N., ter Hofstede, A.H.M., van der Aalst, W.M.P. and Mulyar, N. (2006) *Workflow Control-Flow Patterns: A Revised View*. BPM Center Report BPM-06-22. Available at: https://wfm.ec.tuwien.ac.at/node/46

4. Wikipedia (n.d.) *Workflow engine*. Available at: https://en.wikipedia.org/wiki/Workflow_engine

5. Workflow Management Coalition (1995) *The Workflow Reference Model*. Available at: https://www.aiai.ed.ac.uk/project/wfmc/ARCHIVE/DOCS/refmodel/rmv1-16.html

Task 2

TIVIDY System Scope

TIVIDY is a generic workflow management system which creates, executes and monitors different types of workflows. The system does not have boundaries and is capable of supporting processes such as employee onboarding, procurement, claim approval or document approval.

The workflow manager is responsible for managing workflow definitions, creating workflow instances, coordinating work items, assigning work to participants, reacting to events and failures, maintaining workflow history and communicating with external systems.

The TIVIDY system is responsible for workflow-related behaviour only.

Workflow Definition and Workflow Instance:

A workflow definition describes the structure and rules of a process. It contains information such as the activities that must be performed, the order in which they occur, dependencies between activities, routing rules, required participant roles and possible outcomes.

A workflow instance represents one running execution of a workflow definition. Each instance maintains its own runtime data, current progress, assigned participants, work-item states, decisions, events and execution history.

This separation allows the same workflow definition to create multiple independent workflow instances without changes to one instance affecting the others.

Expected Areas of Variation:

- how work items are assigned to participants;
- how workflow routing decisions are made;
- approval and escalation rules;
- validation and assessment rules;
- external systems used by a workflow;
- notification and monitoring behaviour;
- additional behaviour attached to work items; and
- the structure and ordering of workflow activities.

These work flows are treated as separate entities thus the system does not have to be redesigned to handle the different work flows.

Selected Scenarios: Insurance Claims Workflow

Organisation or Environment

An insurance organisation that receives and processes claims submitted by policyholders. TIVIDY is used to coordinate the work involved in processing these claims from submission until a final outcome is reached.

This work flow is processed by the Generic TIVIDY system and does not define the system.

Process Being Managed

The process begins when a policyholder submits an insurance claim. The claim is then captured and validated before being assigned for assessment. It is then reviewed, after consideration the origination may request the required additional supporting information.

Once sufficient information has been collected, the claim is assessed again. Depending on the outcome, the claim will then be approved, rejected or escalated for further review.

If the claim is approved, payment processing can be initiated through an external payment service. The workflow ends once the claim reaches an appropriate final state and its history has been recorded.

A possible high-level flow is:

Claim submitted → Validate claim → Assign claim → Assess claim → Approve, reject or escalate → Process payment if approved → Close workflow

Participants and Roles

The insurance claim workflow participants:

- Policyholder – submits the claim.
- Claims Administrator – performs initial validation and ensures that all the required supporting documents for the claim is present.
- Claim Assessor – evaluates the claim according to criteria.
- Approver – authorises claims.
- Finance Department – initiates payment for authorised claims.
- Workflow Administrator – monitors workflow execution and handles administrative workflow issues.
- External Services – provide services such as payments.

Major Pieces of Work Activities:

- receiving a claim
- validating claim details and supporting documentation
- classifying the claim
- assigning the claim to a claim assessor
- assessing the claim

- requesting additional information where necessary
- making a decision
- processing approved payments
- closing and recording the completed claims

Important Decisions and Routing Points

Several decisions can affect how the claim is processed.

After submission, the system determines whether the claim contains sufficient information. Incomplete claims may be returned for additional information.

The claim type may determine which assessment rules and Claim assessor are used. The outcome of the assessment determines whether the claim is approved, rejected or escalated.

Claims may also require different approval levels depending on factors such as the value or complexity of the claim.

Approvals and Dependencies

Certain activities can only occur after earlier activities have successfully completed.

- assessment cannot begin until the claim has been validated
- a claim cannot be approved before the assessment is completed
- payment cannot be processed before approval has been granted
- certain claims may require additional approval before payment
- missing information must be supplied before processing can continue.

These dependencies control the order in which work becomes available within the workflow.

Failures and Exceptions

The workflow must be able to deal with exceptional situations such as:

- incomplete or invalid claim information
- missing supporting documents
- unavailable assessors
- failed validation
- rejected claims
- approval timeouts
- claims requiring escalation
- failure of an external service
- failed payment processing.

The system should respond to these situations without requiring changes to the entire workflow.

External Systems

The insurance claim workflow may communicate with external services where required. One important example is an external payment service used to process approved claims.

Other possible integrations may include notification systems, document-storage systems or external verification services.

These systems remain separate from the core workflow engine and are accessed through defined interfaces.

Expected Outcome

The expected outcome is that every submitted claim reaches a controlled final state such as approved and paid, rejected, cancelled or otherwise formally closed.

Throughout the process, TIVIDY should maintain the progress, decisions, state changes and execution history associated with the individual workflow instance. This information can later be used for monitoring, auditing and reporting.

Education - University Application Processing
The basic workflow could be:

1. Student submits an application.
2. Admissions administrator validates the application and supporting documents.
3. If documents are missing, the student is requested to provide them.
4. Application is assigned to an admissions officer.
5. Admissions officer assesses academic eligibility.
6. Application is accepted, rejected or escalated for further review.
7. Final decision is recorded, the student is notified, and the workflow is closed.

Participants: Student, Admissions Administrator, Admissions Officer, Senior Admissions Officer, Admissions Committee and Workflow Administrator.

Possible exceptions: Missing documents, invalid information, unavailable admissions officers, rejected applications and applications requiring escalation.

We can reuse our existing design patterns, including Strategy for assignment, State for work-item lifecycles, Observer for notifications, Chain of Responsibility for escalation, and Adapter for external systems.

For the Google Doc, we would need to add:

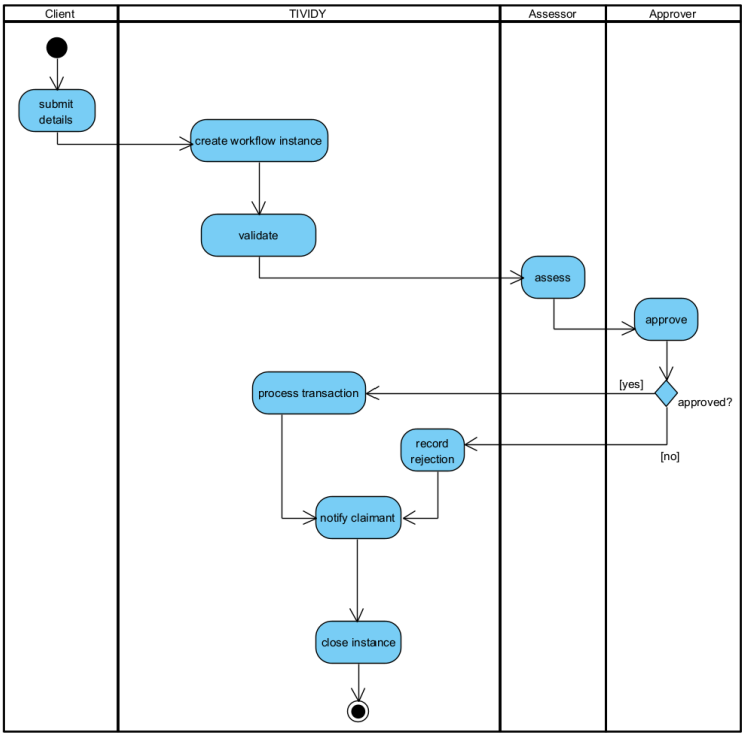
- The second industry's organisation/environment
- Process being managed
- Participants and roles
- Major activities
- Decisions and routing points
- Approvals and dependencies
- Failures and exceptions
- External systems
- Expected outcome

We should also update the system scope to include both industries and add a design revision explaining the introduction of the second industry.

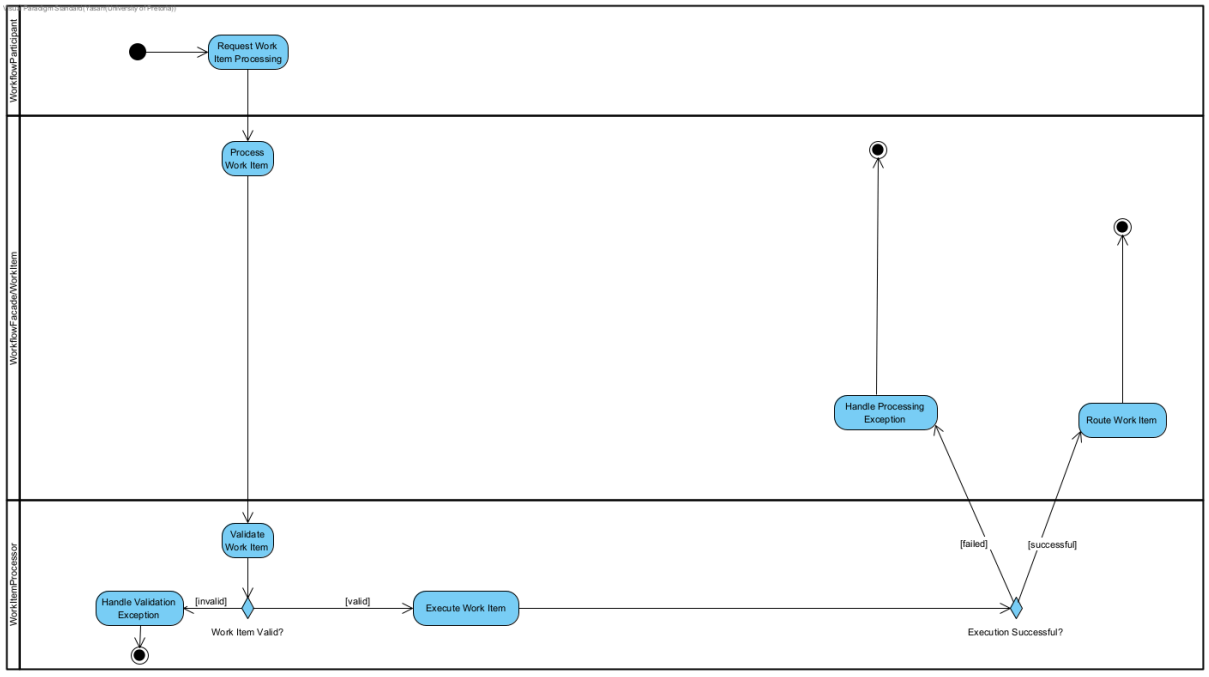
The main idea is that both Insurance and Education use the same generic TIVIDY engine, just with different workflow definitions, participants and rules.

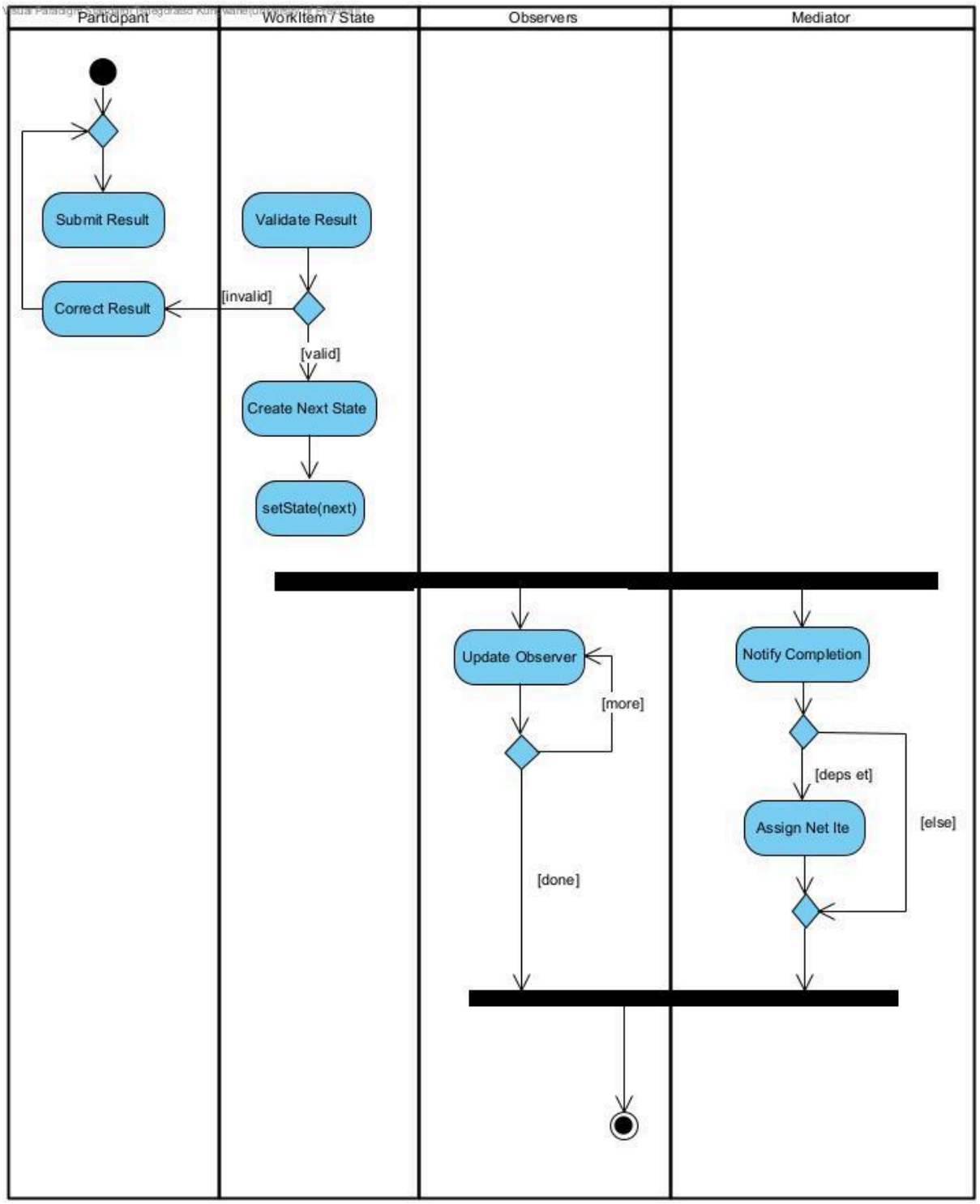
Task 3

Activity diagram showing claim approval



Activity Diagram Showing Work Item Processing and Recovery




Participant
WorkItem / State
Observers
Mediator
Submit Result
Validate Result
Correct Result
[invalid]
[valid]
Create Next State
setState(next)
Update Observer
[more]
[done]
Notify Completion
[deps et]
Assign Net Ite
[else]

Task 4

Observer

Problem: When a WorkItem changes or an important event occurs, multiple interested components may need to react without the WorkItem being tightly coupled to them.

GoF participant	Class	What it does
Subject/ ConcreteSubject	WorkItem	Maintains observers and notifies them when important work-item events occur.
Observer	WorkflowObserver	Defines the common update() interface for objects interested in workflow events.
Concrete Observer	AuditObserver, NotificationObserver, MonitoringObserver	Assigns a work item based on participant roles, current workload, or configurable workflow rules/data.

Collaboration: WorkItem maintains registered WorkflowObservers and notifies them when an important work-item event occurs. Each observer then performs its own response.

What varies: the types of observers and how each observer reacts to work-item events.

Why Observer is appropriate: Observer allows TIVIDY to add or remove notification, auditing, or monitoring components without changing the core WorkItem class.

Adapter (Object adapter)

Problem: TIVIDY may need to communicate with external systems whose interfaces are incompatible with the interface expected by TIVIDY.

GoF participant	Class	What it does
Target	ExternalSystemAdapter	Defines the interface TIVIDY expects when communicating with external systems.
Adapter	ExternalServiceAdapter	Converts TIVIDY requests into calls understood by an external service.
Adaptee	ExternalService	Represents an existing external system with an incompatible interface.
Client	WorkflowMediator	Uses the Target interface when workflow processing requires communication with an external system.

Collaboration: WorkflowMediator communicates through the ExternalSystemAdapter interface. ExternalServiceAdapter translates TIVIDY's request into a form understood by ExternalService.

What varies: The external system being integrated and the interface it provides.

Why Adapter is appropriate: Adapter allows TIVIDY to integrate with different external systems without changing the core workflow logic.

Strategy

Problem: TIVIDY must support different methods of assigning WorkItems to suitable WorkflowParticipants without changing the core workflow engine. Different workflow scenarios may require role-based assignment, workload balancing or configurable eligibility rules. The assignment algorithm must therefore be interchangeable while keeping the assignment process consistent.

GoF participant	**Class**	**What it does**
Strategy	AssignmentStrategy	Defines the common `selectParticipant()` interface for selecting a suitable WorkflowParticipant for a WorkItem.
ConcreteStrategy	RoleBasedAssignmentStrategy, WorkloadAssignmentStrategy, RuleBasedAssignmentStrategy	Selects the first available participant whose role matches the WorkItem's required role. Selects the available participant with the required role who has the lowest current workload. Selects an eligible participant according to role, availability and a configurable maximum workload threshold.
Context	WorkflowMediator	Uses the selected AssignmentStrategy when assigning work.

Collaboration: WorkflowMediator maintains a reference to the selected AssignmentStrategy, which can be configured through setAssignmentStrategy(). When a WorkItem requires assignment, WorkflowMediator calls requestAssignment(), which delegates participant selection to the strategy's selectParticipant() method. The selected concrete strategy evaluates the registered WorkflowParticipants and returns a suitable participant, or nullptr if none qualifies. WorkflowMediator then coordinates the assignment and participant workload updates.

What varies: The algorithm used to select a suitable WorkflowParticipant. TIVIDY supports role-based selection, workload-based selection and configurable rule-based selection without modifying the WorkflowMediator or WorkItem classes.

Why Strategy is appropriate: The Strategy pattern separates assignment algorithms from workflow coordination. This allows different industries and workflow scenarios to use different assignment policies while sharing the same generic engine. New assignment algorithms can be introduced by implementing AssignmentStrategy without changing the existing assignment process.

State

Problem: A WorkItem behaves differently depending on its current lifecycle state. Operations such as making a task available, assigning it, starting it, completing it, approving it, rejecting it and escalating it are only valid under particular conditions. TIVIDY must enforce these lifecycle rules without relying on large conditional statements inside WorkItem.

GoF participant	**Class**	**What it does**
Context	WorkItem	Maintains its current WorkItemState and delegates lifecycle operations to it. Uses `setState()` to replace its current state.
State	WorkItemState	Defines the common interface for lifecycle operations, including `makeAvailable()`, `assign()`, `start()`, `complete()`, `approve()`, `reject()` and `escalate()`. Invalid operations throw exceptions.
ConcreteState	CreatedState, AvailableState, AssignedState, InProgressState, CompletedState, RejectedState, EscalatedState	Implement the lifecycle behaviour permitted in each state and initiate valid transitions.

Collaboration: WorkItem acts as the context and delegates lifecycle operations to its current WorkItemState. Each concrete state determines which operations are permitted and performs the appropriate transition using WorkItem::setState(). For example, CreatedState allows a WorkItem to become available, AvailableState supports assignment to an eligible participant, and AssignedState allows work to begin. InProgressState manages completion, approval, rejection and escalation behaviour. When an operation is not permitted in the current state, the state implementation throws an exception rather than allowing an invalid transition. Assignment also validates the WorkflowParticipant, including availability and required role.

What varies: The permitted operations, lifecycle behaviour and transitions associated with each WorkItemState. Different concrete states implement their own valid operations while sharing the same interface.

Why State is appropriate: The State pattern encapsulates lifecycle-specific behaviour in separate classes instead of placing large conditional statements inside WorkItem. This improves maintainability, makes invalid transitions easier to enforce and keeps the workflow engine extensible. The same WorkItem lifecycle implementation can be reused by different industry workflows without duplicating state-management logic.

Template Method

Problem: Different work items may require different validation, execution, and routing behaviour, while following the same overall processing sequence.

GoF participant	**Class**	**What it does**
AbstractClass	WorkItemProcessor	Defines process() as the template method. It fixes the sequence of validate(), execute() and route(), while declaring these steps as abstract operations for subclasses to implement.
ConcreteClass	DefaultWorkItemProcessor,	Provides the default validation, execution and routing behaviour for WorkItems.
ConcreteClass	CustomWorkItemProcessor	Implements customised validation, execution and routing behaviour for different workflow requirements.
ConcreteClass	ExternalCallProcessor	Processes automatic work items that interact with external systems. It validates request data, invokes an external operation through WorkflowMediator, and completes or rejects the WorkItem based on the response.

Collaboration: A caller invokes process(item) on a WorkItemProcessor. The template method calls validate(item), execute(item) and route(item) in a fixed order. Each concrete processor implements these steps differently.

DefaultWorkItemProcessor performs standard processing, while CustomWorkItemProcessor provides customised behaviour. ExternalCallProcessor uses WorkflowMediator to request an external operation through ExternalSystemAdapter. It then routes the WorkItem based on the external response.

The WorkItem lifecycle is controlled by the State pattern, while WorkflowMediator coordinates when processing is triggered.

What varies: The validation, execution and routing behaviour implemented by each processor. The overall processing sequence remains unchanged.

Why Template Method is appropriate: Template Method provides a consistent work-item processing algorithm while allowing specialised processors to implement different workflow requirements without duplicating the processing sequence. It also allows external-system processing to use the same structure as other workflow activities.

Mediator

Problem: Work items and workflow participants need to coordinate their actions. Direct communication between these objects would spread coordination rules across multiple classes and create tight coupling.

GoF participant	Class	What it does
Mediator	Mediator	Defines notify(originator), the common interface through which colleagues report changes.
ConcreteMediator	WorkflowMediator	Coordinates WorkItems and WorkflowParticipants using separate registered collections. Delegates assignment to AssignmentStrategy, processing to WorkItemProcessor, approvals to ApprovalHandler, and external operations to ExternalSystemAdapter
Colleague	WorkflowColleague	Stores a reference to the Mediator and provides changed(), setMediator() and getMediator() for communicating changes.
ConcreteColleague	WorkItem, WorkflowParticipant	Represent workflow tasks and participants. Notify the Mediator about changes without directly coordinating with each other.

Collaboration: WorkflowMediator registers WorkItems and WorkflowParticipants using registerWorkItem() and registerParticipant(). When a colleague changes, changed() calls notify(this) on its Mediator. The WorkflowMediator identifies the originator and coordinates the appropriate response.

For assignment, it uses AssignmentStrategy to select a suitable participant. When a WorkItem enters the InProgress state, the Mediator invokes the item's WorkItemProcessor or its default processor. WorkItems awaiting approval are passed to ApprovalHandler. When a participant becomes unavailable, the Mediator can escalate and reassign their work items. External operations are handled through ExternalSystemAdapter.

What varies: Assignment strategies, coordination rules, processing behaviour, approval chains, external systems, and responses to workflow events.

Why Mediator is appropriate: Mediator centralises workflow coordination, reducing direct dependencies between WorkItem and WorkflowParticipant. It also integrates other design patterns without requiring coordination logic to be distributed throughout the system.

Facade

Problem:TIVIDY has many interacting classes. Clients should not need to understand or manage this complexity directly.

GoF participant	Class	What it does
Facade	WorkflowFacade	Provides a simple, unified interface to start workflows, complete work items, and query status.
Subsystem Classes	WorkflowDefinition, WorkflowBuilder, WorkFlowInstance, WorkflowMediator, WorkItem, WorkflowParticipant, Command and its subclasses	The complex internal classes that perform the actual work. The facade delegates to them.

Collaboration: Clients call methods on TividyFacade. The facade coordinates the necessary subsystem classes to fulfil the request and returns a simple result.
What varies: The internal implementation and number of classes involved in fulfilling a request can change without affecting clients.
Why Facade is appropriate: Facade hides the complexity of the workflow engine behind a clean interface, making TIVIDY easier to use and reducing coupling between clients and the internal design.

Decorator

Problem:TIVIDY needs to add optional behaviours to WorkItems without modifying the core WorkItem class or creating many subclasses.

GoF participant	**Class**	**What it does**
Component	WorkItem	Defines the common interface for work items.
ConcreteComponent	TaskItem, ApprovalItem, SubprocessItem	The core work item without extra features.
Decorator	WorkItemDecorator	Maintains a reference to a WorkItem and conforms to the same interface.
ConcreteDecorator	PriorityDecorator, DeadlineDecorator, NotificationDecorator, AuditDecorator	Adds specific extra behaviour before or after delegating to the wrapped WorkItem.

Collaboration: A work item is wrapped in one or more decorators, and each decorator adds its own behaviour (such as a notification or an audit entry) before passing the call on to the item it wraps, until the core work item does the real work.

What varies: Which optional behaviours (priority, deadline, notification, audit) a work item has, and in what combination.

Why Decorator is appropriate: It adds features to individual work items without changing the WorkItem classes or needing a subclass for every combination

Chain of Responsibility

Problem: TIVIDY may require a WorkItem to pass through different approval levels. The system must determine which handler can process a request without hard-coding approval rules into the main workflow logic.

GoF participant	**Class**	**What it does**
Handler	ApprovalHandler	Defines the abstract handleRequest() operation and maintains a reference to the next handler through successor. Provides setSuccessor() and forwardRequest() to manage the chain.
ConcreteHandler	FirstLevelApprovalHandler	Each handler decides whether it can process the WorkItem. If it cannot, it forwards the request to its successor.
ConcreteHandler	SecondLevelApprovalHandler	Provides another configurable approval level using its own rule and action, forwarding requests it cannot handle.
ConcreteHandler	FinalApprovalHandler	Provides a final configurable approval level. It performs its action if its rule passes or attempts to forward the request if it does not
Client	WorkflowMediator	Initiates the approval process by passing a WorkItem to the first ApprovalHandler in the chain.

Collaboration: WorkflowMediator starts the approval process by calling handleRequest() on the first ApprovalHandler. Each concrete handler evaluates its ApprovalRule to determine whether it can handle the WorkItem.

If the rule returns true, the handler executes its configured ApprovalAction, which performs the required approval-related behaviour. If the rule returns false, the handler calls forwardRequest() to pass the WorkItem to its successor. If no successor exists, an exception is thrown.

The handlers can be linked using setSuccessor(), allowing the chain to be configured without modifying the WorkflowMediator.

What varies: The number and order of handlers, the approval rules used by each handler, and the actions performed when an approval rule succeeds.

Why Chain of Responsibility is appropriate: Chain of Responsibility separates approval decisions from the core workflow logic. TIVIDY can configure different approval levels for insurance claims, university applications or other workflows by changing the rules, actions and order of handlers rather than modifying the main workflow engine.

Builder

Problem: Creating a WorkFlowInstance can require assembling multiple parts together, like the definitions, participants, WorkItems and data. Making these directly causes the creation logic to become overly complex and difficult to maintain.

GoF participant	**Class**	**What it does**
Builder	WorkFlowBuilder	Defines the steps required to construct a WorkflowInstance.
ConcreteBuilder	InsuranceClaimWorkflowBuilder	Implements the construction steps for an insurance-claim workflow instance.
Director	WorkFlowDefinition	Controls the order in which the builder steps are performed.
Product	WorkFlowInstance	The complex object being created.

Collaboration: WorkflowDefinition uses a WorkflowInstanceBuilder to construct a WorkflowInstance. The builder assembles the required workflow information, participants, and work items before returning the completed WorkflowInstance through build().

What varies: The way workflow instances are assembled and the components included in different workflow types.

Why Builder is appropriate: Builder separates the construction of a complex WorkflowInstance from its representation, allowing TIVIDY to support different workflow configurations without changing the creation process.

Command

Problem: Workflow actions such as assigning, completing, rejecting, or escalating a work item should be treated as separate objects so that they can be executed, recorded, and potentially undone without changing the core workflow logic.

GoF participant	**Class**	**What it does**
Command	Command	Defines the common execute() and undo() interface for workflow actions.
ConcreteCommand	AssignCommand, CompleteWorkCommand, RejectCommand, EscalateCommand	Encapsulate specific actions performed on a WorkItem object.
Receiver	WorkItem	Performs the actual work when a command is executed.
Invoker	WorkFlowMediator	Executes commands and coordinates workflow behavior
Client		Creates and submits commands for execution.

Collaboration: A WorkflowMediator creates a Command and invokes execute(). The command forwards the request to the associated WorkItem, which performs the actual operation. Different command objects encapsulate different workflow actions while presenting the same interface.

What varies: The workflow action being performed, such as assigning, completing, rejecting, or escalating a work item.

Why Command is appropriate: Command encapsulates workflow actions as separate objects, allowing TIVIDY to execute, record, manage, and extend workflow operations without modifying the core workflow engine.

Factory Method

Problem: TIVIDY has different types of work items: tasks, approvals and subprocesses. Each type must be created differently, but if/else and switch statements are not allowed (Rule 6). Each workflow instance needs its own copy of the data

GoF participant	**Class**	**What it does**
Creator	WorkItemCreator	The abstract creator
ConcreteCreators	TaskItemCreator, ApprovalItemCreator, SubprocessItemCreator	Implements concrete creators
Product	WorkItem	The abstract products
ConcreteProducts	TaskItem, ApprovalItem, SubprocessItem.	The different product types that are created by the various factories.

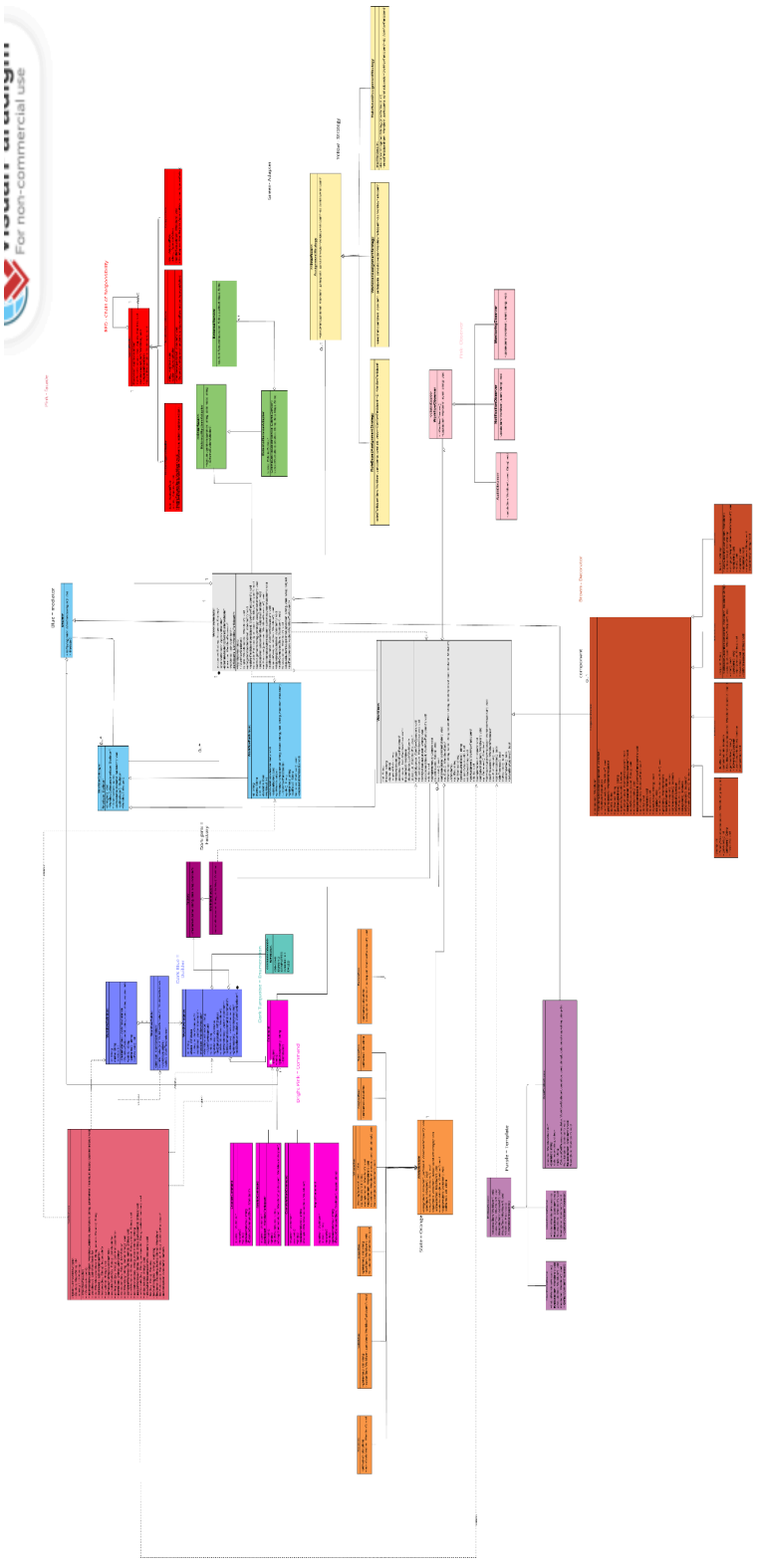
Collaboration: The system reads a step and finds the correct creator from a registry It calls creator.instantiate(step) The creator creates the correct type of WorkItem The WorkItem is given its data, starts in CreatedState, and gets an AuditObserver The client then uses the WorkItem through the WorkItem interface

What Varies: The type of WorkItem being created: task, approval or subprocess

Why Factory Method Is Appropriate:

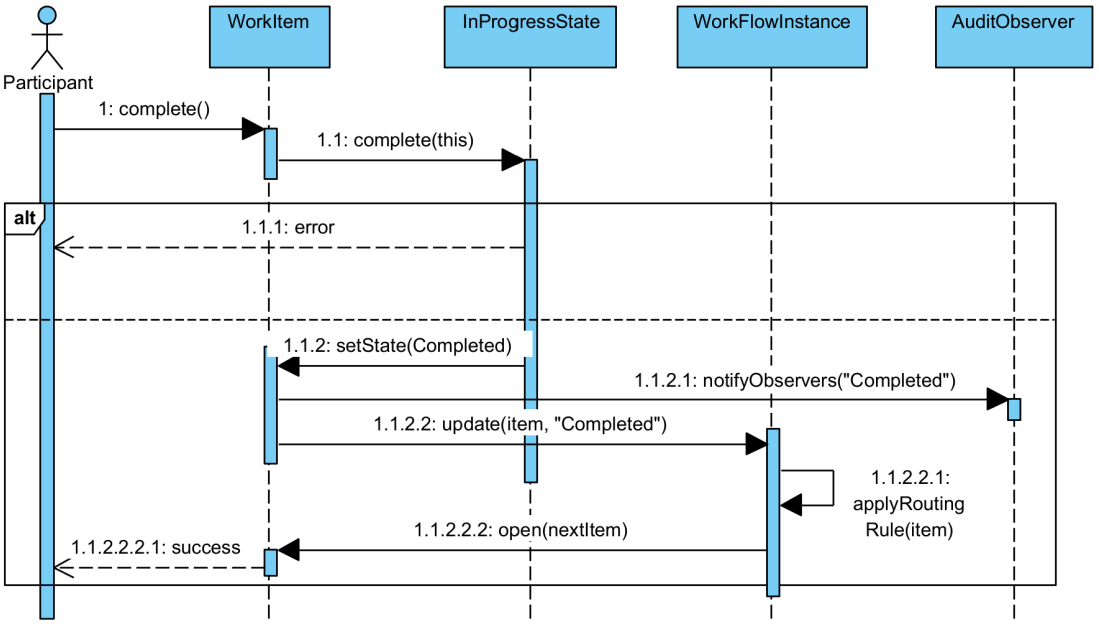
- Each type of WorkItem has its own creator
- A new type can be added by creating a new product and creator
- No changes to the workflow engine are needed
- The common setup is kept in one place, so every WorkItem is created with the correct state and observer

Task 5 – UML

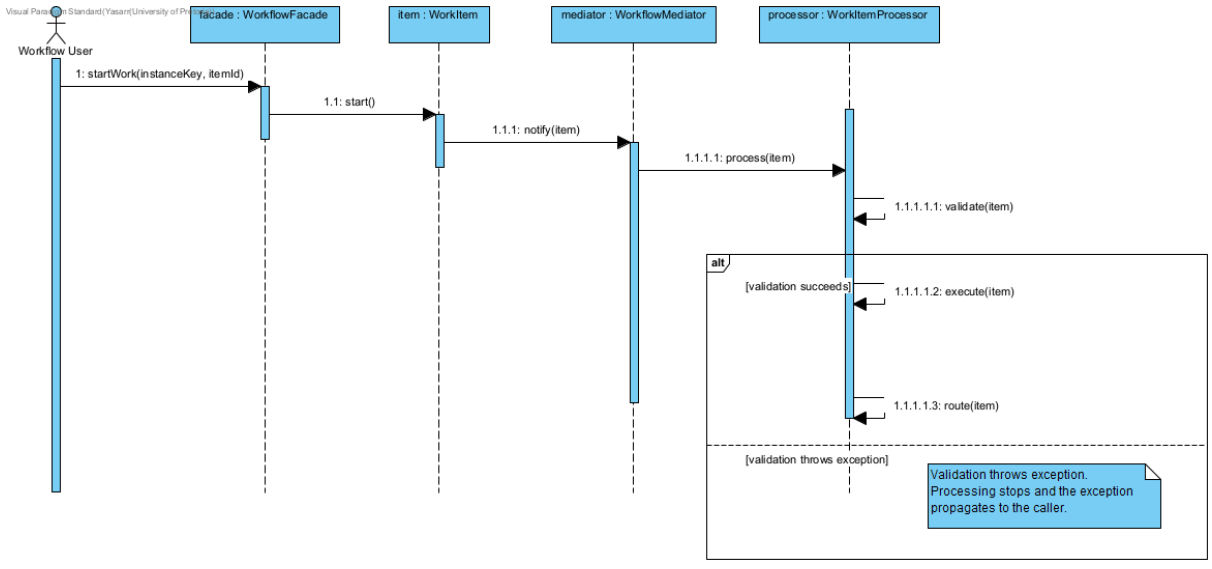


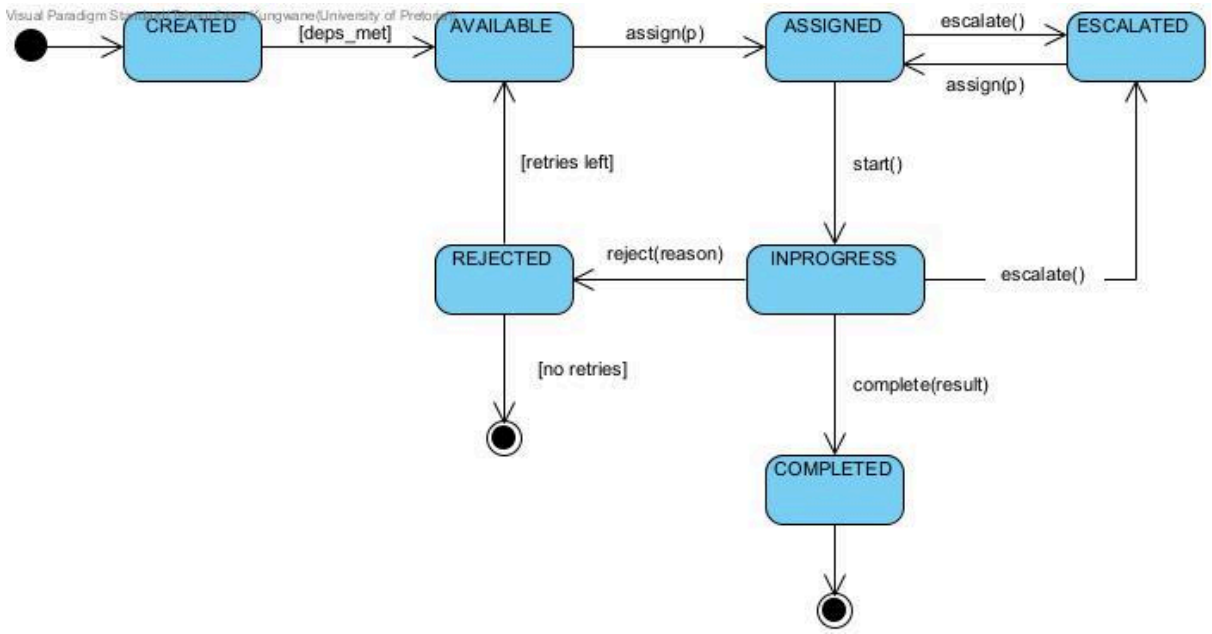
Task 6

Sequence diagram showing completing a work item



Sequence Diagram Showing Work Item Submission and Initial Processing




CREATED
[deps_met]
AVAILABLE
assign(p)
ASSIGNED
escalate()
ESCALATED
assign(p)
[retries left]
start()
REJECTED
reject(reason)
INPROGRESS
escalate()
[no retries]
complete(result)
COMPLETED

Task 7- Design decision history

Revision 1: Generalising the Workflow System

Initially, TIVIDY was designed around insurance claim processing. However, we revised the architecture to create a generic workflow engine that could support multiple industries. This allows different workflows to reuse the same core functionality without modifying the engine.

Revision 2: Introducing a Second Industry

We introduced University Application Processing alongside Insurance Claims Processing to demonstrate the flexibility of our system. Both industries use the same engine but have different participants, activities, approval rules and outcomes.

Revision 3: Integrating Design Patterns

We integrated the GoF design patterns into the workflow engine to improve flexibility and maintainability. Patterns such as State, Strategy, Mediator and Chain of Responsibility manage different responsibilities, reducing dependencies between classes and allowing functionality to be extended more easily.

Revision 4: Introducing a Graphical User Interface

We introduced a browser-based GUI to make the system easier to use and demonstrate. The interface communicates with the C++ engine through GuiApi and WorkflowFacade, allowing users to create workflows, manage work items, perform actions and monitor workflow progress.

Revision 5: Automated Testing and Engineering Improvements

We added automated testing using doctest to verify important workflow functionality and interactions between components. We also introduced Docker to provide a consistent execution environment and GitHub Actions to automate building, testing and integration checks. These improvements helped ensure the system remained reliable as development progressed.

Updated functional requirements

TIVIDY was initially designed around insurance claims but was later generalised to support workflows across multiple industries. The final system uses a reusable C++11 workflow engine that manages workflow instances, work items, participants, approvals and routing independently of industry-specific requirements.

The following functional requirements reflect the final implementation.

Functional Requirement	**Implementation / Design Pattern**	**Related Tests**
The system shall create and manage independent workflow instances.	WorkFlowDefinition, WorkFlowInstance, Builder	WorkflowInstance
The system shall manage work-item lifecycles and prevent invalid state transitions.	WorkItem, WorkItemState, State	State
The system shall assign work items using role-based, workload-based and rule-based strategies.	AssignmentStrategy, Strategy	Strategy
The system shall coordinate assignments, reassignment and participant availability.	WorkflowMediator, Mediator	Mediator
The system shall support configurable approval, rejection and escalation processes.	ApprovalHandler, Chain of Responsibility	Approval
The system shall manage workflow routing, dependencies and cancellation	WorkFlowInstance, WorkflowMediator	WorkflowInstance
The system shall notify relevant components of workflow events and maintain an audit trail.	WorkflowObserver, Observer	Not separately listed
The system shall communicate with	ExternalSystemAdapter, Adapter	Not separately listed

external services through a common interface.		
The system shall execute workflows from different industries using the same engine.	Industry builders, Builder	Integration
The system shall allow users to execute and monitor workflows through a graphical interface.	GuiApi, WorkflowFacade, GUI	Manual GUI testing
The system shall support workflow actions and maintain command history, including undo where supported.	Command, WorkflowFacade	Not separately listed
The system shall allow users to design and save custom workflows.	Workflow designer, JSON storage	Not separately listed

Implementation explanation

1. The engine (includes/ and src/). This is generic and knows nothing about any industry. A workflow definition describes a type of workflow, and every run is an independent WorkFlowInstance with its own work items, participants, mediator and history. Each WorkItem moves through a lifecycle using the State pattern: Created, Available, Assigned, InProgress, Completed, Rejected or Escalated. The WorkflowMediator coordinates everything. When an item changes, it frees workload on completion, starts the approval chain when approval is needed, runs the item's processor when work starts, and picks the next person with the assignment strategy. When a person becomes unavailable, it escalates their items and reassigns them. Approvals go through a Chain of Responsibility, external services through an Adapter, user actions are Command objects kept in a history, and optional features such as priority, deadlines, notifications and auditing are Decorators. WorkflowFacade gives the whole engine one simple entry point.

2. The industries (app/Industries.cpp). Everything industry-specific lives here: the participants, work items, approval rules, routing rules and external-service data. Each industry is a concrete builder. WorkflowDefinition acts as the Director and calls the builder's steps in order to create an instance. Nothing in the engine is changed or subclassed to support an industry.

3. The GUI (app/gui/ and gui/). A small C++ web server serves a web page. Every button click becomes a request to GuiApi, which turns it into calls on WorkflowFacade. The page only draws what the engine returns. It does not simulate any workflow. The page can start instances, show current work, take actions (such as start, complete, reject, escalate, assign, undo, cancel), mark people away, and show the audit trail as well as command history. There is also a workflow designer, so a user can describe a new workflow (people, steps, approval limits, routing) and run it on the same engine. Designs are saved as JSON in data/custom_workflows.json.

Graphical user interface

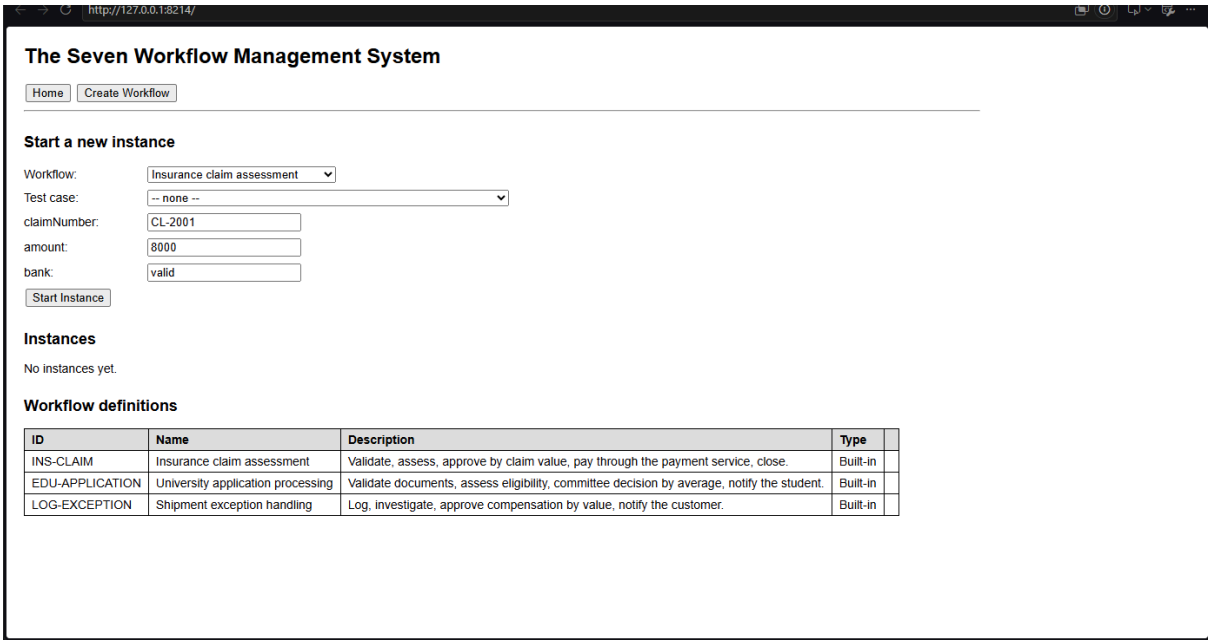
TIVIDY is driven through a browser GUI that calls the real C++11 engine for every action; the browser holds no workflow logic. The program `tividy_gui` starts the engine, serves the GUI on port 8214 and opens it in the default browser (`make gui`).

A click travels left to right: the page sends a JSON request, `GuiApi` turns it into Facade calls, and the engine's reply comes back the same way. Saved designs go through the custom workflow library, which registers them with the Facade.

How it is built. The C++ side has three small parts, all in `app/gui/`:

- `HttpServer` is a minimal socket server written for the project (POSIX sockets and Winsock), so no third-party library is needed. It handles one request at a time, so the engine is never called from two threads.
- `GuiApi` turns each request into `WorkflowFacade` calls and returns JSON. After every action it compares the instance before and after the call and reports what the engine did: state changes, routing steps added, the new status.
- `gui_main.cpp` registers the three built-in industries, loads saved custom workflows and starts the server.

The page itself (`gui/index.html`, `gui/app.js`, `gui/style.css`) only displays what the engine returns. Because every button goes through the Facade, invalid actions are refused by the engine's State classes and the reason is shown to the user, for example "Cannot complete work item in state Created".



Custom Workflows and Added Work Items

Users can design new workflows and add work items to running instances without changing any engine code; both reuse the existing patterns.

Custom workflows. On the Create Workflow page a user defines input fields, participants and roles, ordered work items (task, approval or subprocess), the assignment strategy, the processor, approval limits and routing rules. `CustomWorkflowLibrary` (`app/CustomWorkflow.cpp`) validates the design, builds a trial instance to catch mistakes early, and registers it with the Facade like a

built-in industry. Designs are saved to `data/custom_workflows.json` and loaded again at start-up.

Added work items. On a running instance a user can add a work item after the current work. `WorkflowFacade::insertWorkItem` refuses an item placed before the current work, a duplicate ID or a role nobody has. Otherwise the item joins the normal flow: it opens when the item before it completes, and the instance's strategy assigns it.

Pattern	**Where it is used**
Builder	`ConfigurableBuilder` builds an instance from a user's design; `WorkflowDefinition` is the Director, as for the built-in industries
Factory Method	New work items are made through `CreatorRegistry` and its creators
Decorator	`assembleWorkItem` (`WorkItemSpec`) wraps items in Priority, Notification, Deadline and Audit decorators as the user chooses
Strategy	The design picks the workload, role or rule based assignment strategy
Chain of Responsibility	Approval limits become the three approval handlers
Template Method	The design picks the default or custom processor; automatic steps use `ExternalCallProcessor`
Adapter	Automatic steps call the external service through

	`ExternalServiceAdapter`
Facade	`insertWorkItem` and `removeWorkflow` were added to `WorkflowFacade`, so the GUI still only talks to the Facade

Testing and Results

Automated Testing Framework

TIVIDY uses the **doctest C++ testing framework** to verify the correctness and reliability of important workflow-management functionality. Automated unit and integration tests are used to check workflow state transitions, participant assignment, approvals, routing, instance isolation and interactions between system components.

All test source files are located in the `tests/` directory, while compiled test executables are generated in `build/tests/`.

The Makefile automatically discovers, compiles and executes test files matching `tests/*Tests.cpp`.

Test Coverage

The project includes 10 automated test source files covering the following functionality:

Test file	Functionality tested
`StateTests.cpp`	Work-item state transitions and invalid operations
`StrategyTests.cpp`	Role-based, workload-based and rule-based assignment
`MediatorTests.cpp`	Participant assignment, reassignment and coordination
`ApprovalTests.cpp`	Approval, rejection and handler-chain behaviour
`WorkflowInstanceTests.cpp`	Workflow progression, routing, instance isolation and cancellation
`WorkflowIntegrationTests.cpp`	Integration across insurance claims and university applications

`RoutingRuleTests.cpp`	Workflow routing conditions and decisions
`FacadeTests.cpp`	Workflow operations exposed through the Facade
`CustomWorkflowTests.cpp`	Custom workflow functionality
`AdHocWorkItemTests.cpp`	Ad hoc work-item functionality

Running Automated Tests

The automated tests require a C++ compiler, GNU Make and the doctest development package.

On Ubuntu or WSL, dependencies can be installed using:

sudo apt update

sudo apt install build-essential doctest-dev

To compile and execute all automated tests from the project root directory:

make test

Alternatively, tests can be executed using the previously built TIVIDY Docker image:

docker run --rm tividy214 make test

Test Results

The project README records the following results from 31 October 2026:

33/33 tests passed

123/123 assertions passed

0 failures

The same successful results were recorded when executing the automated tests through Docker.

The Docker image, scripted demonstration and browser-based GUI were also reported as successfully tested.

These results provide evidence that the tested workflow functionality operated as expected during the recorded execution. The automated tests also support regression detection when changes are made to the workflow engine.

Testing Contribution

Automated testing was implemented to verify both individual design-pattern behaviour and the interactions of the reusable workflow engine. Particular attention was given to the State and Strategy patterns, ensuring that work-item lifecycle operations and participant-selection algorithms behave according to their defined rules.

CI/CD pipeline (bonus)

A GitHub Actions pipeline (.github/workflows/ci.yml) builds and tests TIVIDY on every push and pull request to dev and main, and packages a release build when main changes. No change reaches those branches without the whole suite being run on a clean machine.

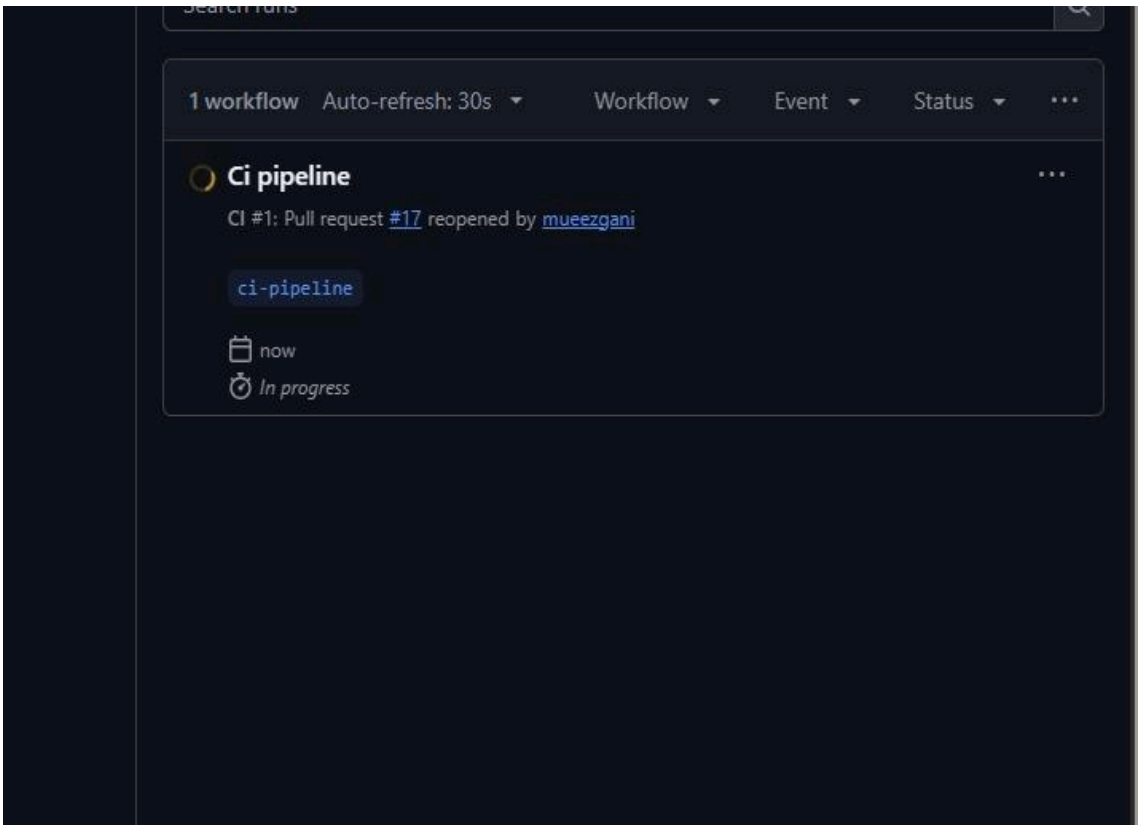
The four checks run side by side on every push and pull request; packaging runs only for main, and only when all four have passed.

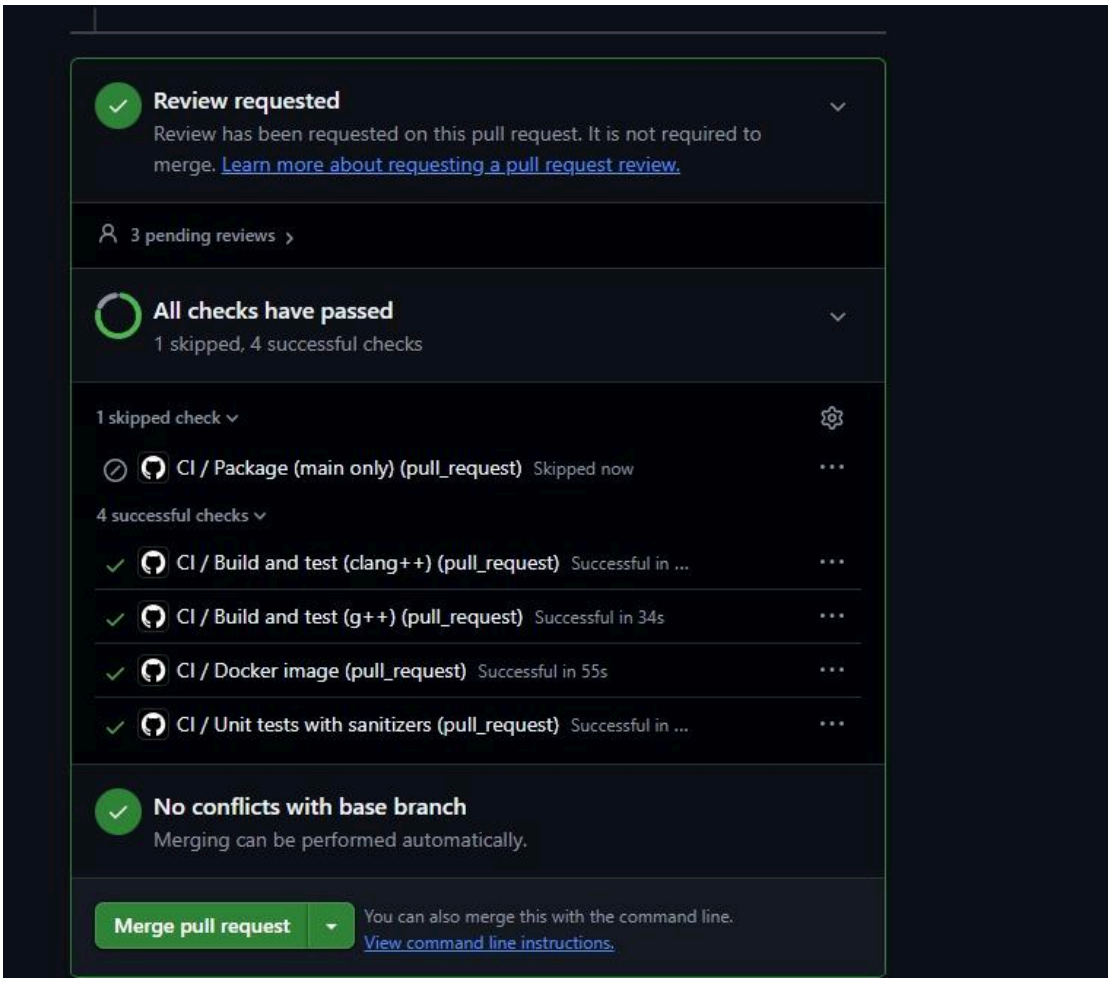
Job	When	What it does
Build and test (g++)	every push and pull request to dev or main	Installs doctest, builds with warnings treated as errors, runs make test, the scripted demo and the GUI smoke test, uploads JUnit test reports
Build and test (clang++)	same	The same steps with a second compiler, to catch code that only one compiler accepts

Unit tests with sanitizers	same	Runs the suite with AddressSanitizer and UndefinedBehaviorSanitizer to catch memory errors, leaks and undefined behaviour
Docker image	same	Builds the team's build/Dockerfile, runs make test and make demo inside the container and checks that the GUI answers on port 8214
Package (main only)	push to main, after all four jobs pass	Builds an optimised release and uploads tividy-linux.tar.gz (GUI, demo, web files, README) as a download

Why it matters. A broken build or failing test is caught on the pull request, before a reviewer approves it, so dev and main always build. The two compilers and the sanitizers find faults a single local build misses. The Docker job proves the container a marker might run still works. The package job is the delivery step: every change to main produces a ready-to-run build.

Evidence. The pipeline's first run, on pull request #17, passed all four jobs (g++ in 34 s, Docker image in 55 s). The Actions tab of the repository keeps every run with its logs, test reports and packaged build.


1 workflow
Auto-refresh: 30s
Workflow
Event
Status
Ci pipeline
CI #1: Pull request #17 reopened by mueezgani
ci-pipeline
now
In progress


Review requested
Review has been requested on this pull request. It is not required to merge. Learn more about requesting a pull request review.
3 pending reviews
All checks have passed
1 skipped, 4 successful checks
1 skipped check
CI / Package (main only) (pull_request) Skipped now
4 successful checks
CI / Build and test (clang++) (pull_request) Successful in ...
CI / Build and test (g++) (pull_request) Successful in 34s
CI / Docker image (pull_request) Successful in 55s
CI / Unit tests with sanitizers (pull_request) Successful in ...
No conflicts with base branch
Merging can be performed automatically.
Merge pull request
You can also merge this with the command line.
View command line instructions.

Build, run and test instructions

The project uses C++11 and GNU Make.

The following dependencies are required:

- g++ compiler with C++11 support.
- GNU Make.
- doctest for automated testing.
- Docker Desktop or Docker Engine for containerised execution.
- A web browser for the GUI.

Building the project

From the repository root directory, run

make - builds project using makefile.

Running the GUI

To launch the graphical interface:

make gui

Running the Scripted Demonstration

make demo

The scripted demonstration provides a repeatable way to demonstrate project functionality.

Running Automated Tests

make test

This command compiles and runs all six automated test suites.

Running TIVIDY Using Docker

Docker provides an Ubuntu-based environment containing the required compiler, GNU Make and doctest dependencies.

The Docker configuration includes:

- `build/Dockerfile` - installs dependencies and builds the application.
- `.dockerignore` - excludes unnecessary files from the Docker build.

Build the Docker image:

```
docker build -f build/Dockerfile -t tividy214 .
```

Run the GUI through Docker:

```
docker run --rm -p 8214:8214 tividy214
```

Open the following address in a browser:

http://localhost:8214

Press `Ctrl+C` to stop the container.

Run the scripted demonstration:

```
docker run --rm tividy214 make demo
```

Run automated tests:

```
docker run --rm tividy214 make test
```

The Docker image, GUI, demonstration and tests were successfully executed on 31 October 2026.

GitHub and engineering evidence

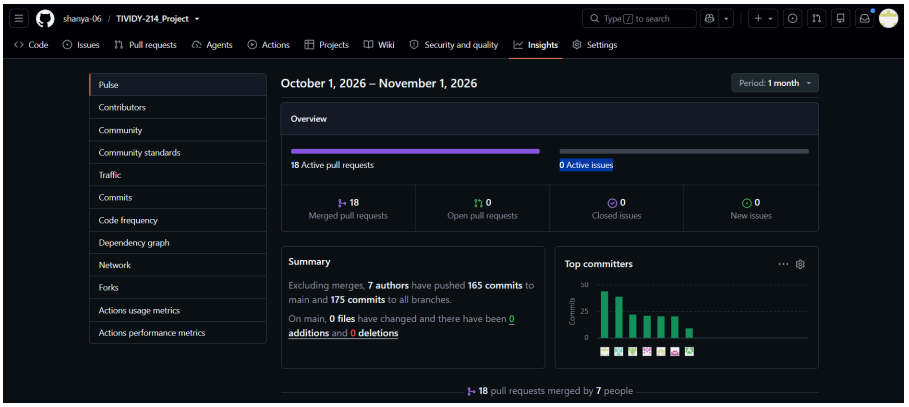
https://github.com/shanya-06/TIVIDY-214_Project

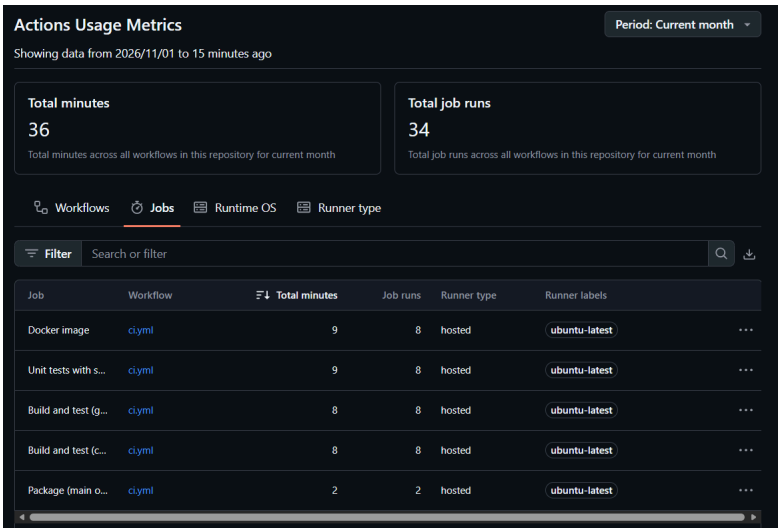
During development, GitHub was used as a centralised repository, allowing all team members to access, contribute to, and manage the project from a shared location. It provided version control, facilitated collaboration through branching and pull requests, and ensured that the latest project changes were available to the entire team.

All major development took place in a development branch, from which each team member created separate feature branches for their assigned work. These changes were then integrated through pull requests.

Pull requests required multiple reviews before being merged, ensuring that team members validated and reviewed each other's code before it was added to the central branch. This process helped maintain code quality, encouraged knowledge sharing, and reduced the likelihood of integration issues.

Once development and testing were complete, the development branch was merged into the main branch, which contains the final version of the TIVIDY system.





Team contribution statement

UML Class Diagram:

- **Tamima:** Observer, Adapter, Strategy
- **Mohammed:** Mediator, Template Method, State
- **Mueez:** Decorator, Facade, Chain of Responsibility
- **Shanya:** Builder, Command, Factory

Activity & Runtime Diagrams:

- **Tshego:** State Diagram and one Activity Diagram
- **Nasiha:** One Activity Diagram and one Sequence Diagram
- **Yasar:** One Activity Diagram and one Sequence Diagram

Research: Nasiha and Mueez contributed to the workflow-management research.

Implementations:

- **Yasar:** State, Strategy
- **Mueez:** Chain of Responsibility, WorkItem
- **Mohammed:** Mediator, Template Method
- **Shanya:** Builder, Command
- **Tamima:** Observer, Adapter
- **Tshego:** Factory
- **Nasiha:** Decorator, Facade

Additional Features

- **Yasar:** Unit testing
- **Tamima:** Docker, Doxygen
- **Tshego & Shanya:** PowerPoint presentation
- **Mueez:** CI/CD pipeline, GUI
- **Mohammed:** GUI

All members contributed to reviewing and integrating the final TIVIDY design.